

DL for ENG

## LECTURE 6.2

# Post-Training and Reinforcement Learning

Training a model from scratch is now the unusual case. What you do before, after and instead — and how to make the result run fast enough to steer a car that is already moving.

**Remus Teodorescu**

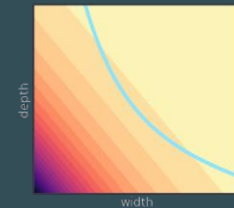
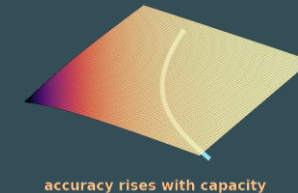
*With support from Research Assistant Noman Khan - nomank@energy.aau.dk*

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

*transfer · post-training · reinforcement learning · imitation · deployment*

## THE LIFE OF A MODEL



the blue line is  
30 frames per second

past it, a model is  
accurate and useless

and so does latency

*The last lecture was the middle of this list and nothing else. Everything above and below it is today, and the last two rows are the ones with no textbook.*

# Learning Objectives

## SECTIONS, IN ORDER

- 1 **Transfer learning** — starting from trained weights; fine-tuning; self-supervised pre-training
- 2 **Post-training** — the three stages of a 2026 model, and where the reinforcement learning sits
- 3 **Reinforcement learning** — what it is; return and value; credit assignment; policy gradient
- 4 **Imitation learning** — behavioural cloning, when to prefer it, and what the camera car does
- 5 **Deployment** — the latency budget; quantisation and its cost; pruning; compilation
- 6 **Challenges** — what goes wrong first, and what to check

## WHAT WE SKIP, AND WHERE IT IS TAUGHT

|                          |                                             |
|--------------------------|---------------------------------------------|
| <b>evaluation</b>        | overfitting, bias and variance — L4.2       |
| <b>loss, optimisers</b>  | the last lecture — L6.1                     |
| <b>the camera car</b>    | vision-based navigation — L11.1             |
| <b>generative models</b> | not in this course; a word to read the news |

*Deployment has no textbook. Its slides cite vendor documentation and say so.*

# Recap

## THE TEN QUESTIONS OF L6.1 — LOSS FUNCTIONS AND GRADIENTS

- 1** What is a loss function, where does it come from, and what is the five-step recipe? *UDL ch. 5.1 – 5.2*
- 2** What distribution does squared error assume, and what happens to that fit when a sensor spikes? *UDL ch. 5.3, 5.8; Ex\_06 notebook 01*
- 3** Derive cross entropy from a categorical distribution; what should an untrained ten-class model report? *UDL ch. 5.5 – 5.7*
- 4** Why does plain gradient descent zig-zag in a narrow valley, and what does momentum change? *UDL ch. 6.1 – 6.3; Ex\_06 notebook 02*
- 5** What two averages does Adam keep, what does each fix, and why is its loss not monotone? *UDL ch. 6.4; Kingma & Ba (2015)*
- 6** What does L-BFGS use that Adam does not, why is it full-batch, and when is the hand-off worth it? *GBC ch. 8.6, 4.2; Ex\_06 notebook 02*
- 7** What does backpropagation compute, why does it cost one extra pass, and why does training need more memory? *UDL ch. 7.2 – 7.4*
- 8** Why does a zero initialisation fail, and what do the He and Glorot rules keep constant? *UDL ch. 7.5*
- 9** Why can autograd differentiate with respect to an input, and how does that make a loss with no data? *Ex\_02 notebook 02; Part 2*
- 10** What is the weight  $\lambda$  between two loss terms, physically, and what must a report say about it? *Ex\_06 notebook 01; Part 2 reports*

# Transfer Learning

## WHAT IT IS, WHY IT WORKS, WHAT IT SAVES

- **what** — initialise from a network trained on a large dataset
- **why** — its first layers learn edges and textures, not its task
- **how** — keep the early layers, replace the head, retrain the end
- **the saving** — a few hundred images instead of a hundred thousand
- **applications** — weld seams, radiographs, any small labelled set
- the most reliable trick in applied deep learning

### WHAT EACH DEPTH HAS LEARNED

|              |                           |
|--------------|---------------------------|
| layer 1      | oriented edges            |
| layer 2      | corners, colour blobs     |
| layers 3 – 4 | textures, repeated motifs |
| layers 5 – 6 | object parts              |
| the head     | the thousand class names  |

*Only the last row is about the original task. That is the row you throw away and replace with your own — two classes, or one steering angle.*

# Fine-Tuning

## THREE SETTINGS, AND HOW TO CHOOSE

- **freeze everything, train the head** — least data, the first try
- **unfreeze the last blocks** — at a rate ten to a hundred times smaller
- **unfreeze everything** — plenty of data and a different domain
- **choose on similarity to the pre-training data** — not on set size
- **catastrophic forgetting** — fine-tune too hard and the features go

**DECIDE · 2 MIN** Two hundred labelled images of a domain unlike the photographs the backbone saw. Which row of the table on the right do you choose, and at what learning rate?

### WHAT TO UNFREEZE

| setting       | data                  | domain           |
|---------------|-----------------------|------------------|
| head only     | small                 | similar          |
| last blocks   | medium                | the usual answer |
| everything    | large                 | different        |
| learning rate | 10 – 100 ×<br>smaller | always           |

### CATASTROPHIC FORGETTING

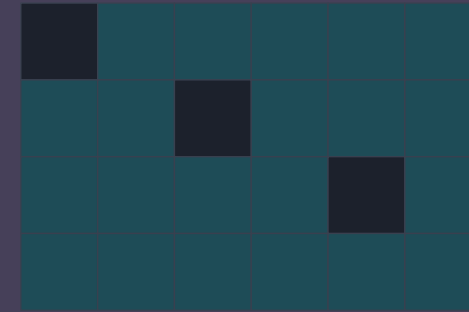
Fine-tune too hard and the network loses the general features it was carrying. The cure is the small learning rate, and freezing the early layers.

# Self-Supervised Pre-Training

## LABELS MADE FROM THE DATA ITSELF

- transfer needs somebody to have trained the original
- **self-supervision** — invent a task whose answer is in the raw data
- **hide part of the input and predict it** — nobody labels anything
- images: mask patches; text: hide a word; traces: the next second
- **for you** — years of unlabelled turbine data, a few labelled faults

### MASK, THEN PREDICT



three patches hidden — the  
network predicts them

*The three dark squares are the label. Nobody wrote it down, and a new mask gives a different one.*

**images**

mask patches

**text**

predict the next word

**sensor traces**

predict the next second

# Post-Training

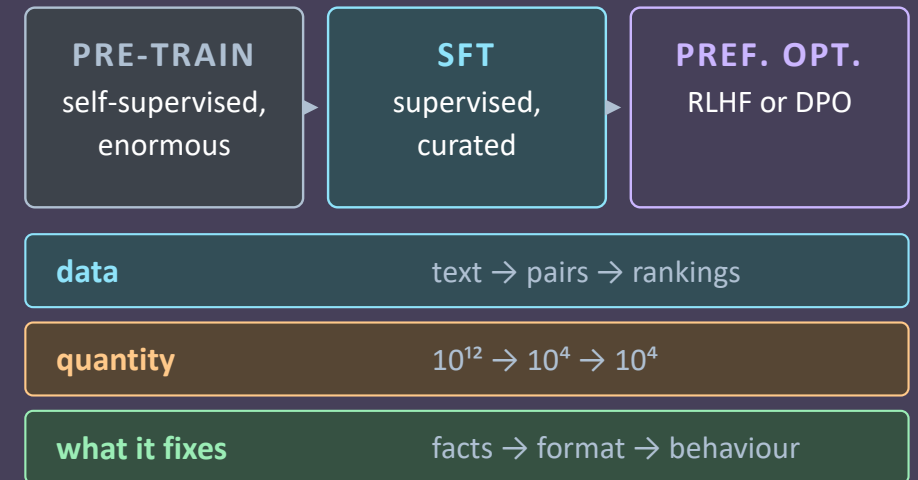
## THREE STAGES, THREE NAMES

You will not build any of them in this course, and no generative modelling is taught here — but the words are unavoidable and worth ninety seconds.

## PRE-TRAIN, THEN FINE-TUNE, THEN ALIGN

- **pre-training** — self-supervision at enormous scale
- **supervised fine-tuning** — instruction pairs, supervised
- **preference optimisation** — no correct answer, a comparison
- **the third stage is reinforcement learning** — a late scalar
- the first time in this course there is no target

### THE THREE STAGES



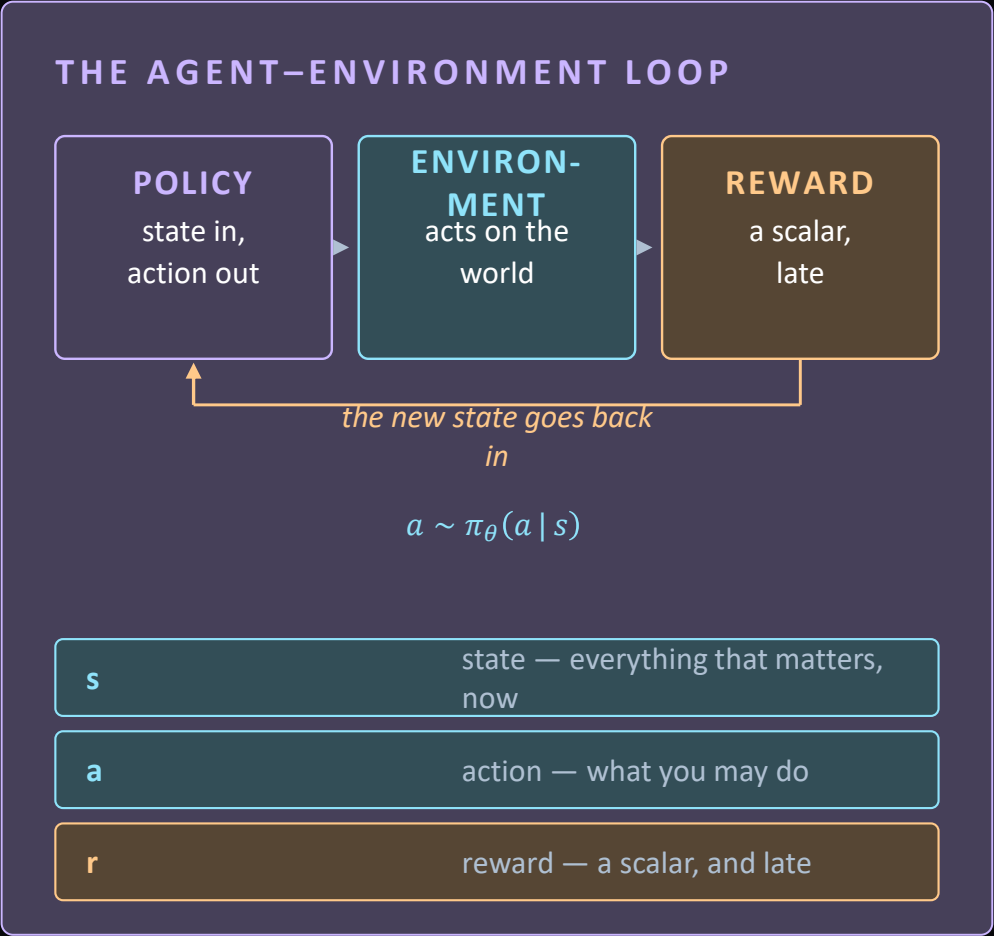
# Reinforcement Learning

## KIND, FUNCTION, ARCHITECTURE, APPLICATIONS

- **kind** — a learning problem, not an architecture: act, consequence
- **function** — a policy: state in, action out, to maximise the return
- **architecture** — the agent–environment loop; Markov states
- **applications** — control, scheduling, dispatch, games, alignment

## AGAINST SUPERVISED LEARNING, AND AGAINST CONTROL

|            | supervised         | reinforcement           | control                |
|------------|--------------------|-------------------------|------------------------|
| the signal | a target per input | a scalar, late          | a cost per step        |
| the model  | of the mapping     | of the policy           | of the plant           |
| the world  | not needed         | a simulator, or samples | a plant model          |
| guarantees | held-out error     | none                    | stability proofs       |
| vocabulary | x, y               | s, a, $\pi$ , r, G, V   | x, u, controller, cost |



# Return and Value

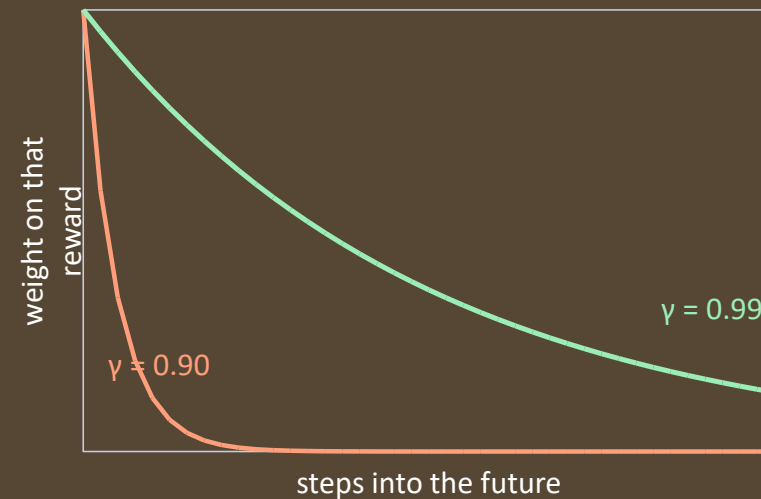
## THE OBJECTIVE IS A SUM OVER THE FUTURE

- the agent maximises the return — not the next reward
- the sum of all rewards from here on, discounted by  $\gamma$
- $\gamma$  keeps the sum finite, and says how much the future matters
- the value of a state — the return you expect from it
- read  $\gamma$  as a time constant — set it from your horizon

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

$G_t$  the return from step  $t$ ,  $r$  one reward,  $\gamma \in (0, 1)$  how far ahead you look

## HOW FAR AHEAD $\gamma$ LOOKS



$$Q^\pi(s, a) = \mathbb{E}[G_t \mid s_t = s, a_t = a]$$

$\gamma = 0.90$  is effectively blind beyond fifty steps. Choose it from your system's timescale.

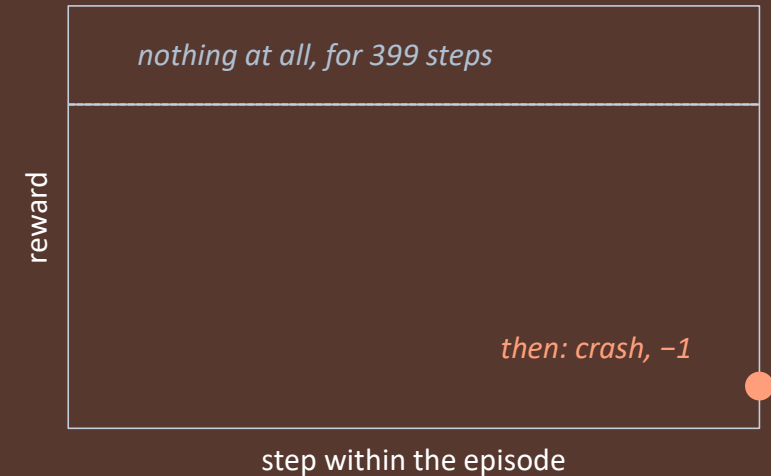
# Credit Assignment

## ONE LATE NUMBER, FOUR HUNDRED DECISIONS

- an episode is four hundred steps and one number at the end
- **something in those four hundred was responsible** — which
- **credit assignment** — why RL is hard rather than merely slow
- **the blunt approach** — credit every action equally, many episodes
- **the refinements** — discounting, learned values, advantage

**TALK · 2 MIN** In the plot on the right the reward is zero for 399 steps and minus one at the crash. Which of the 400 actions do you blame, and what evidence would you need to be sure?

## ONE REWARD, FOUR HUNDRED ACTIONS



*A sparse reward, and it is the normal case. Shaping it — small rewards along the way — helps a great deal and is also how you accidentally teach the wrong task.*



# Policy Gradient

## A GRADIENT WITHOUT DIFFERENTIATING THE WORLD

- the policy is a network, so it has gradients — the world does not
- make the policy stochastic — a distribution over actions
- an action's probability is differentiable — the world need not be
- run an episode, record what was done and what followed
- raise the probability of actions followed by good returns

## THE POLICY GRADIENT

$$\nabla_{\theta} J = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a | s) G_t]$$

*Raise the probability of what worked, in proportion to how well it worked: the maximum likelihood recipe with the outcome as a weight.*

## WHAT IT COSTS

|                  |                                |
|------------------|--------------------------------|
| <b>variance</b>  | enormous — hence the baselines |
| <b>samples</b>   | millions of episodes is normal |
| <b>stability</b> | no guarantee of any kind       |

# Imitation Learning

## BEHAVIOURAL CLONING

Record a human doing the task, then fit a network to predict the action they took from what they saw. Supervised regression, not reinforcement learning.

## REINFORCEMENT LEARNING

Needs: a reward you can compute, very many episodes, so usually a simulator, and nothing that can be damaged while it explores.

Buys: it can exceed the best demonstration, and solve tasks nobody knows how to demonstrate.

Costs: sample-hungry, unstable, no guarantees, and you get exactly what you reward — reward hacking is where most projects quietly go wrong.

## IMITATION LEARNING

Needs: somebody who can do the task, for as long as it takes to record a few thousand examples.

Buys: ordinary supervised learning, so the loss curve, the validation split and early stopping all apply unchanged.

Costs: it cannot beat the demonstrator, and off the demonstrated distribution it has no idea what to do, and the error compounds.

$$\hat{\theta} = \operatorname{argmin}_{\theta} \sum_i (\pi_{\theta}(s_i) - a_i)^2$$

$\hat{\theta}$  the fitted policy,  $(s_i, a_i)$  one state and the action a human took, no reward anywhere

*A simulator and no expert: reinforce. An expert and an expensive world: imitate.*

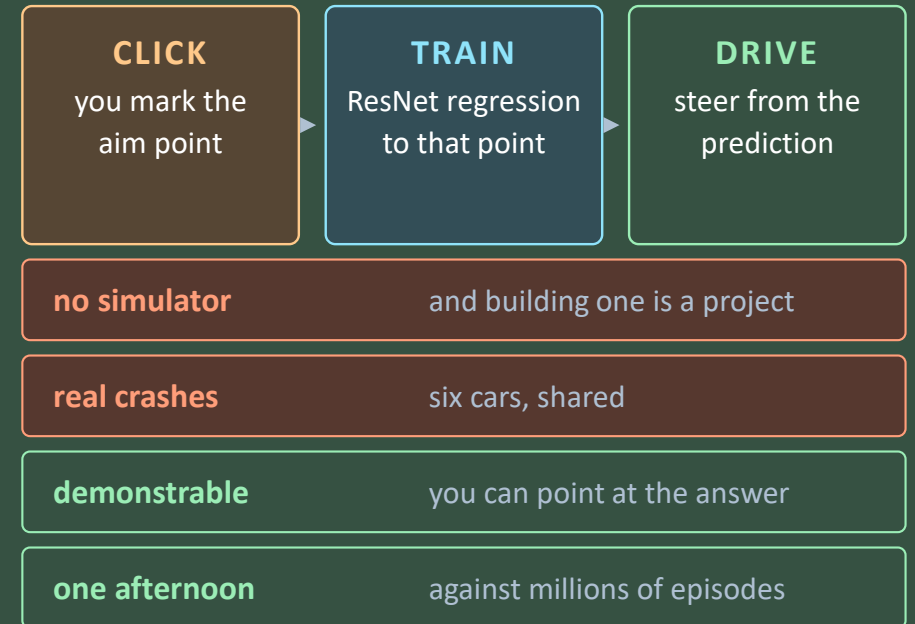
# Steering by Imitation

## WHAT THE CAMERA CAR DOES, AND WHY

- **the car does behavioural cloning** — not reinforcement
- the loss is squared error, and the target is your steering
- no simulator, real crashes, a demonstrable task, an afternoon
- **what you give up** — compounding error off the recorded path
- so the exercise has you drive the recovery cases on purpose

**TALK · 2 MIN** In the pipeline on the right the label is a clicked aim point. What situations will the recorded laps never contain, and what happens to the car the first time one occurs?

### THE STOCK PIPELINE



### WHAT DECIDES IT

Latency, not accuracy. That is the next section.

# Latency Budget

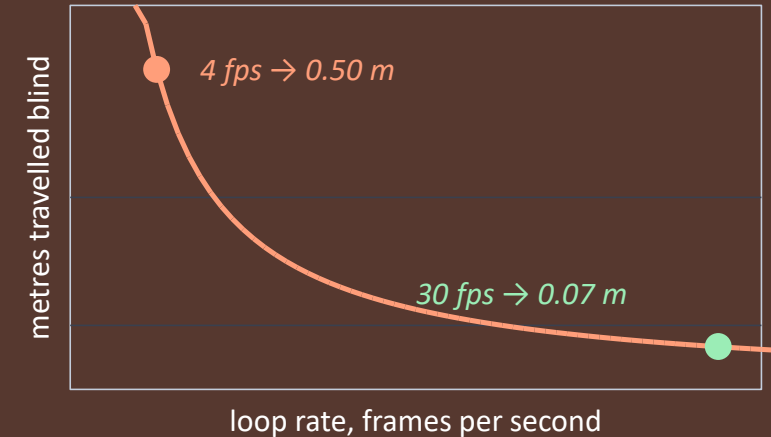
## ON A MOVING CAR, LATENCY IS DISTANCE

- **a model that is accurate and too slow** — does not work
- **the car moves at  $v$ , the loop takes  $t$**  — it travels  $v \cdot t$  blind
- **two metres per second at four frames** — half a metre
- **at thirty frames** — seven centimetres, and the car behaves
- **nothing about the model changed** — only how often it was asked
- **the sources** — vendor documentation; neither textbook covers this

$$d = v / f$$

*Blind distance. Declare it, in metres, in the report.*

### BLIND DISTANCE AT 2 m/s



### THREE TECHNIQUES, THEY COMPOSE

|              |                       |
|--------------|-----------------------|
| quantisation | fp32 → int8           |
| pruning      | delete whole channels |
| compilation  | ONNX, TensorRT        |

# Quantisation

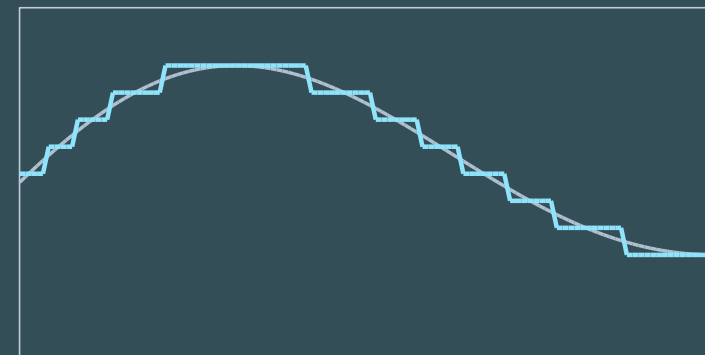
## FP32 TO INT8: THE SAME ARITHMETIC AS AN ADC

- **a trained weight is a 32-bit float** — few bits carry information
- **map the range onto 256 levels** — a scale and an offset
- **four times smaller** — and integer arithmetic is faster
- **the error** — uniform rounding, bounded by half a level
- **post-training conversion is nearly free** — aware training, a retrain

$$q = \text{round}(x/s) + z \quad \hat{x} = s(q - z)$$

$x$  the float value,  $q$  the integer stored,  $s$  the scale,  $z$  the zero point

### A SIGNAL AT THREE BITS



### WHAT IT BUYS

|                       |                                    |
|-----------------------|------------------------------------|
| <b>size</b>           | 4 × smaller                        |
| <b>memory traffic</b> | 4 × less — usually the binding one |
| <b>arithmetic</b>     | integer, and faster                |

# Quantisation Costs

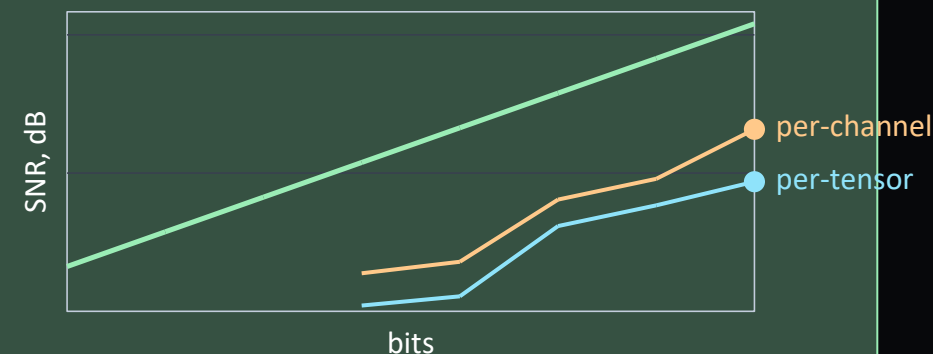
## THE SAME CODE, THE SAME BIT WIDTH, TWO MODELS

A regression's error grows two hundredfold. A classifier loses nothing at all. Quote the classifier and deploy the regression, and a true number has misled you.

### MEASURED IN EX\_06 NOTEBOOK 04

- **classification barely moves** — regression error moves a lot
- **1.6e-06 to 3.5e-04** — over two hundred times worse
- **an argmax absorbs a perturbation** — a regression reports it
- **the steering network is a regression** — the pessimistic case
- per-channel scales buy a factor of eight, for 30 extra floats

### SIGNAL-TO-NOISE AGAINST BIT WIDTH



$$s = \frac{x_{\max} - x_{\min}}{255} \quad \text{Var}(e) = \frac{s^2}{12}$$

$x_{\max} - x_{\min}$  the range, 255 levels,  $s$  the step

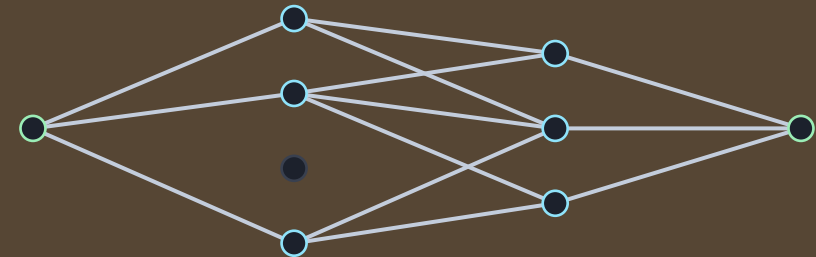
# Pruning

## DELETING WHAT IS NOT USED

- rank the weights by magnitude, zero the smallest, fine-tune
- **unstructured** — high sparsity, no speedup on ordinary hardware
- **structured** — whole channels go; less sparsity, real speedup
- **on a small GPU** — prune structurally, or do not bother
- measure wall-clock latency, not the sparsity figure

**PREDICT · 1 MIN** You zero half the weights scattered through the network on the right. What happens to the wall-clock time on an ordinary GPU, and what would you remove instead?

### STRUCTURED, NOT SCATTERED



one channel removed — the network is simply narrower

**unstructured** high sparsity, no speedup

**structured** lower sparsity, real speedup

**what to measure** latency, not parameter count

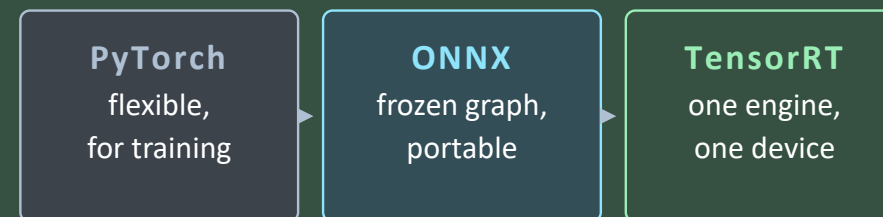
*The greyed node and its edges are gone. Because a whole channel went, the remaining matrices are smaller rather than sparser — the only version an ordinary GPU can exploit.*

# Compilation

## ONNX AND TENSORRT

- **PyTorch is built for flexibility** — it rebuilds the graph every pass
- **ONNX** — a frozen graph, framework-independent
- **TensorRT** — fuses operations and picks kernels for one GPU
- the speedup is often several-fold, for no accuracy change
- **the engine is device-specific** — build it on the car, not the laptop

### THE TOOLCHAIN



*torch2trt shortcuts the middle step.*

### WHAT THE COMPILER DOES

|                         |                                  |
|-------------------------|----------------------------------|
| <b>layer fusion</b>     | conv + bias + ReLU as one kernel |
| <b>kernel selection</b> | benchmarked on the actual GPU    |
| <b>precision</b>        | fp16 or int8 where it can        |

# Challenges

## FINE-TUNING MADE IT WORSE

catastrophic forgetting · the rate was too large, or too much was unfrozen · divide the rate by ten, freeze everything but the head, then work back up

## THE AGENT LEARNED THE WRONG TASK

reward hacking: it did exactly what you asked · no simulator, an expert available, or something that can be damaged: do not reinforce · write the reward down and ask a colleague how to cheat it

## QUANTISED, AND THE ACCURACY HELD

check which metric · an argmax hides what a regression cannot · measure what you care about, in physical units: degrees of steering, not loss

## DATA AND ENERGY EFFICIENCY

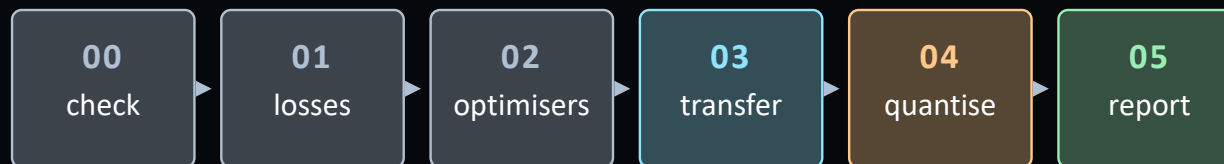
|                  |                                      |
|------------------|--------------------------------------|
| <b>free</b>      | measure the loop rate, do not assume |
| <b>free</b>      | compute the blind distance           |
| <b>free</b>      | state which metric you quoted        |
| <b>cheap</b>     | freeze more, lower the rate          |
| <b>cheap</b>     | compare int8 against fp32 directly   |
| <b>real work</b> | collect recovery data                |

*The last row is the imitation-learning failure, and it costs a session: a model trained only on good driving has never seen the situation it must get out of.*

# Exercise 6

SIX NOTEBOOKS; 03, 04 AND 05 ARE THIS LECTURE'S

- **00** — environment check, and the L-BFGS closure convention
- **01** — the loss recipe, by hand and then in PyTorch
- **02** — four optimisers on one problem; the measured figure
- **03** — transfer learning on a small image set, then fine-tune
- **04** — float32 to int8, on a classifier and then on a regression
- **05** — the report, built from the files 01 to 04 write



## THE QUESTIONS 03 AND 04 ANSWER

- 03** with thirty labels, do scratch, probe and fine-tune separate at all?
- 04** what does int8 cost a classifier, and what does it cost a regression?
- 05** a latency budget — measured, not assumed

## REPORT QUESTION

Thirty labels barely separate scratch, probe and fine-tune. Which model would you deploy, and what would have to be true for that to be right?

# Key Takeaways

## 1 Transfer learning — the usual case

Start from pre-trained weights, replace the head, unfreeze a little at a tenth of the learning rate, and choose on similarity of domain, not size of data. Self-supervision, hiding part of the input and predicting it, is where those weights came from.

## 2 Post-training — three stages

Pre-train by self-supervision at enormous scale, fine-tune on instruction pairs, then align on preferences. The third stage has no correct answer, only a late comparison, and that is reinforcement learning.

## 3 Reinforcement learning — a narrow tool

A learning problem, not an architecture: state, action, policy, reward, return, value, which map onto control's state, input, controller and cost. It earns its cost with a simulator and no demonstrator; credit assignment is why it is hard, and you get exactly what you reward.

## 4 Imitation learning — the camera car

Behavioural cloning is supervised regression from what a human saw to what they did. No simulator, real crashes, a demonstrable task and one afternoon of data decide it. The price is compounding error, so collect recovery data on purpose.

## 5 Deployment — latency is distance

int8, structured pruning and TensorRT are the difference between 4 and 30 frames per second, and at 2 m/s that is 50 cm of blindness against 7. Quantisation costs a classifier nothing and a regression two hundredfold; measure the metric you care about.

*Next — L7.1: the optimisers of the last lecture, pointed at a physics residual.*

# Questions

## TEN QUESTIONS FOR THE ORAL EXAMINATION

- 1 What is transfer learning, why do the first layers transfer, and when should you unfreeze more than the head? *UDL ch. 9.3; Ex\_06 notebook 03*
- 2 What task does self-supervised pre-training invent, and what does it buy an engineer with unlabelled data? *UDL ch. 9.3*
- 3 What are the three stages of post-training, and why is the third one reinforcement learning? *UDL ch. 19; [ext]*
- 4 What kind of problem is reinforcement learning, what is the policy, and how do its objects map onto control? *UDL ch. 19.1; Sutton & Barto (2018)*
- 5 What is the return, why discount, and how would you set  $\gamma$  for a system with a known timescale? *UDL ch. 19.2*
- 6 What is credit assignment, why does a late aggregate reward make learning hard, and what is reward hacking? *UDL ch. 19.3*
- 7 How does the policy gradient obtain a gradient from a world you cannot differentiate? *UDL ch. 19.4 – 19.5*
- 8 Imitation or reinforcement for the camera car: what decides it, and what does imitation give up? *Ex\_11.1; Ross et al. (2011)*
- 9 Why does int8 quantisation barely hurt a classifier and badly hurt a regression, and which does the car run? *Ex\_06 notebook 04; [ext]*
- 10 A model runs at four frames per second on a car at 2 m/s. How far does it travel blind, and what are the three ways to shorten that? *Ex\_11.1; [ext]*

NEXT

# L7.1 — Physics-Informed Neural Networks

---

Part 1 ends here. Everything the last lecture taught about optimisers is about to be pointed at a physics residual: the same Adam, the same L-BFGS, and no data in the loss at all.

## BEFORE THE NEXT SESSION

|             |                                    |
|-------------|------------------------------------|
| read        | L6.1, physics in the loss, again   |
| run         | Ex_06 notebooks 03, 04 and 05      |
| bring       | your int8 comparison, both metrics |
| think about | what your residual would be        |

*Six blocks of general deep learning are finished. From L7 every network you build has an equation in its loss, and the three physics-in-the-loss slides are the only preparation you need.*